

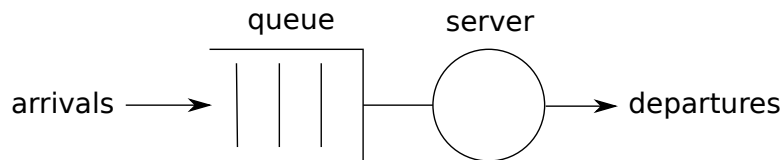
Computer Engineering 4DK4

Lab #1

Performance of Single Server Queueing Systems

Queueing systems occur when “customers” arrive to a system to obtain “service”. When there is limited service capacity, the customers may be allowed to “queue up” and wait their turn before they can obtain the service that they require. Real-world queueing systems are almost everywhere. At a grocery store, for example, shoppers (customers) often must wait in line (i.e., queueing up in first-come-first-served order) until the cashier (the server) is free before obtaining service. In computer networks, customers may consist of data packets that arrive to an Internet router and must be buffered (queued up) while waiting for transmission on an outgoing data link (the server). This process may incur delays that affect the quality of the service that the network can provide. Knowledge of queueing system performance is therefore an important key to understanding the behaviour of packet switched networks such as the Internet.

This lab is an introduction to the performance of single-server queueing systems. A program written in C is used to investigate the behaviour of single server queueing systems where customers with fixed service times arrive according to a Poisson process. A single server queue is often drawn as follows.



Customers arrive to the system at the left, wait in the queue until the server is free and eventually enter the server for a time duration known as their service time. Eventually the customer departs the system on the right. The C program is very simple and does not distinguish between customers, but merely keeps track of time, the total number of customers in the system, and various other information. Based on this, it determines whether the next event to occur is a customer arrival or a customer departure.

1 Purpose

The purpose of this lab is to explore the performance of M/D/1 and M/M/1 queueing systems through structured experimentation. You will investigate how delay, utilization, and queue length behave under different traffic intensities. Particular emphasis is placed on understanding the importance of the stability condition and observing how system behavior changes as traffic intensity approaches and exceeds one.

2 Preparation

1. It is important that you attend the lectures that introduce the C program to be used. You will be responsible for knowing how the simulator works.
2. An electronic copy of the simulation program must first be obtained from the course web site. The name of the program is `coe4dk4_lab_1.c`. You must also obtain a copy of the `simlib.c`, `simlib.h` and `trace.h` library files that you will compile and link together with `coe4dk4_lab_1.c`. Everything needed is zipped together in a single file, `coe4dk4_lab_1.zip`, available on the course web site.
3. Set `SERVICE_TIME = 1 + (the last digit of one team member's student ID)`. This is fixed for the entire lab.
4. Familiarize yourself with running the simulation program. When the simulation finishes a run, it writes output on the screen. There are various parameters such as the customer `ARRIVAL_RATE`, `SERVICE_TIME` and `NUMBER_TO_SERVE` which you can set near the top of the simulation. For each change in the simulation you must recompile and run the program. Once you are familiar with compiling and running the program, continue on and perform the following experiments.¹

3 Experiments

1. The offered traffic load of a queueing system is defined as $\rho = \text{mean arrival rate} \times \text{mean service time}$. For a single-server infinite-buffer system, the condition $\rho < 1$ is required for **stability**. When ρ is close to 1 but still less than 1, the system operates in the **near-critical region**. When $\rho \geq 1$, the system is **unstable or overloaded**.

For a **stable system, steady-state** performance measures such as mean delay, mean number (of customers) in the system, and server utilization converge to constant values as simulation time increases. A queueing system is in the **steady state** if its statistical behavior no longer changes with time. A single-server infinite-buffer system never enters the steady state if $\rho \geq 1$.

In the code provided, `ARRIVAL_RATE` represents the mean arrival rate and `SERVICE_TIME` represents the mean service time. Unless otherwise stated, the `SERVICE_TIME` is fixed. You will vary the `ARRIVAL_RATE` to change the offered traffic load. `NUMBER_TO_SERVE` controls the simulation run length.

Begin with a large value of `NUMBER_TO_SERVE`, e.g., 10^6 , and observe how the performance metrics, including mean delay, mean number (of customers) in the system, and server utilization, vary across different `RANDOM_SEED` values for systems operating in **clearly stable**, **near-critical**, and **unstable** regions. Define your own boundary between the clearly

¹Rather than manually performing each simulation run, you may prefer to wrap the provided code in loops while changing the various parameters. The multiple runs can then be automated.

stable and near-critical regions. Then reduce `NUMBER_TO_SERVE` to 10^4 and 10^2 , and again examine the variability across `RANDOM_SEED` values in each of the stability regions.

Based on your observations, answer these questions:

- i) Under what conditions are results sensitive to the random seed?
 - ii) Under what conditions are results sensitive to the simulation run length?
 - iii) For a clearly stable system, how can you determine whether the reported metrics represent steady-state performance?
 - iv) For a system whose parameters are unknown, is it possible to infer its stability based on simulation output?
2. In this part, consider only **stable configurations** ($\rho < 1$).

Generate a plot of mean delay vs. `ARRIVAL_RATE`. *The value for mean delay that you plot should be the average of the values you obtain for different random generator seeds at a particular value of `ARRIVAL_RATE`.*

Answer these questions:

- i) Explain the reason for the mean delay value at low `ARRIVAL_RATE` values, i.e., What is the mean delay axis intercept in terms of the other system parameters?
 - ii) Explain the behaviour of the mean delay as the `ARRIVAL_RATE` increases and the offered traffic load approaches 1. What is the value of the `ARRIVAL_RATE` axis asymptote when this happens? Explain the shape of the mean delay curve obtained.
 - iii) Can you collect the variance of the delay experienced by individual customers using the given code, why?
3. Modify the program so that instead of M/D/1 you simulate an M/M/1 queuing system, i.e., instead of Poisson process arrivals and deterministic service times, the system now has Poisson arrivals and exponentially distributed service times with the same mean of `SERVICE_TIME` i.e., when the program needs a service time,
- ```
exponential_generator((double)SERVICE_TIME);
```
- will return an exponentially distributed service time with the proper mean<sup>2</sup>.
- Plot curves to study the performance. Plot the mean delay vs. `ARRIVAL_RATE` for the M/D/1 and M/M/1 cases on the same graph and explain the differences and similarities.
4. Repeat Part 3 but this time set the `SERVICE_TIME` to be double of the value in Part 3 and vary `ARRIVAL_RATE` over a *suitable range*. Note that the suitable range should cover the **clearly stable** and **near-critical** regions, and avoid the **unstable** region. Compare the mean delay, mean number of customers, and server utilization vs. `ARRIVAL_RATE` curve to what you obtained before. Compare the curves with the ones from Part 3 and explain the differences. Instead of using the `ARRIVAL_RATE` as the x-axis, if you change the x-axis to be  $\rho$ , how does it affect your observations?

---

<sup>2</sup>Read a description of the M/M/1 queue at [http://en.wikipedia.org/wiki/M/M/1\\_queue](http://en.wikipedia.org/wiki/M/M/1_queue).

5. Using Queueing Theory it can be shown that the mean delay, including both the queueing delay and service time, for a work conserving single server queueing system with Poisson process arrivals and a general service time distribution (i.e., an M/G/1 queue) is given by<sup>3</sup>

$$\bar{d}_{M/G/1} = \bar{X} + \frac{\lambda \bar{X}^2}{2(1 - \rho)} \quad (1)$$

where  $\bar{X}$  is the mean service time (SERVICE\_TIME),  $\rho$  is the product of the mean arrival rate and the mean service time, i.e.,  $\rho = \lambda \bar{X}$ ,  $\lambda = \text{ARRIVAL\_RATE}$ , and  $\bar{X}^2 = \sigma_X^2 + \bar{X}^2$  is the second moment of the service time<sup>4</sup>. Note that the total delay of a customer is the sum of its service time and its queueing delay. Therefore, the two terms in Equation 1 are the mean service time and the mean queueing delay incurred by the customers, respectively.

When the service times are fixed (i.e., an M/D/1 queue, as in the supplied simulation),  $\sigma_X = 0$ , and therefore, the mean delay can be written as

$$\bar{d}_{M/D/1} = \frac{\bar{X}(2 - \rho)}{2(1 - \rho)} \quad (2)$$

When the service times are exponentially distributed,  $\sigma_X^2 = \bar{X}^2$ ,  $\bar{X}^2 = \sigma_X^2 + \bar{X}^2 = 2\bar{X}^2$  and therefore, the mean delay can be written as

$$\bar{d}_{M/M/1} = \frac{\bar{X}}{1 - \rho} \quad (3)$$

Compare the results that you plotted above to this analytic result. (As you can see that the mean delay formulas only apply to systems when  $\rho < 1$ .)

6. Modify the code so that it simulates a single server queueing system with a maximum queue size, e.g., define MAX\_QUEUE\_SIZE at the top of the simulation. When a customer arrives and finds that the number in the queue is at its maximum, the arriving customer is rejected from the system without being served. At the end of each simulation run, output the customer rejection probability, i.e., the ratio of the number of rejected customers to the total number of customer arrivals. Note that only served customers are included in the mean delay calculation.

In this type of system, there is no limit on the arrival rate that can be used, since customers that find a full queue will simply be rejected, i.e., the queue can never grow larger than MAX\_QUEUE\_SIZE. For this reason,  $\rho < 1$  does not have to be true.

Without running your simulation, answer the following questions:

- i) What is the maximum delay a customer may experience?
- ii) What is the maximum rejection rate (or fraction) of customers?

<sup>3</sup>Read a description of the M/G/1 queue at [https://en.wikipedia.org/wiki/M/G/1\\_queue](https://en.wikipedia.org/wiki/M/G/1_queue).

<sup>4</sup>Equation 1 is known as the Pollaczek-Khinchine formula. See [https://en.wikipedia.org/wiki/Pollaczek-Khinchine\\_formula](https://en.wikipedia.org/wiki/Pollaczek-Khinchine_formula).

For various values of `MAX_QUEUE_SIZE`, plot the mean customer delay versus arrival rate and the rejection fraction versus the arrival rate.

Choose appropriate values of `MAX_QUEUE_SIZE` and suitable ranges of arrival rates so that you can observe both the mean customer delay and the rejection fraction change asymptotically from near zero to their respective maximum values. Discuss the differences and similarities between the infinite and finite queue cases. For which parameter settings was it most difficult to generate mean delay or rejection fraction curves that span the full range from near zero to their theoretical maximum values? Explain why.

## 4 Writeup

Each group (maximum of three students) should submit a single writeup for the lab. The writeup should clearly document your experiments, results, observations, and answers to the questions posed in Section 3.

1. Experimental setup: Describe the parameter values used in each experiment, including the arrival rates, service times, simulation run lengths, queue sizes (where applicable), and random number generator seeds. Clearly indicate which parameters were varied and which were held fixed.
2. Results and plots: Include all plots requested in Section 3. Unless otherwise indicated, choose parameter ranges that adequately illustrate the behavior of the system from low to high traffic load. Clearly label all axes, curves, and relevant parameter values. When multiple curves are being compared, present them on the same graph where appropriate.
3. Discussion: Explain the important features and trends observed in your results rather than simply describing the shapes of the curves. Address all questions posed in Section 3 and relate your observations to the queueing concepts discussed in the lab, including stability, steady-state behavior, traffic load, and finite versus infinite queue capacity.
4. Analytical comparison: Where analytical results are provided, compare your simulation results with the corresponding theoretical predictions. Discuss any noticeable differences and possible reasons for them.
5. Simulation reliability: Where applicable, discuss the effects of simulation run length and random number generator seed on your results. Make sure that the simulation settings used to generate your final performance curves provide sufficiently reliable results.
6. Code modifications: Include a listing of the portions of the supplied simulation code that you modified. You do not need to include unchanged portions of the supplied code.
7. Performance curves should show the underlying simulation results clearly. Do not use piecewise-linear interpolation or overfitted curves to connect or smooth the simulation data.